

Rapport du Projet Spring Boot/Thymeleaf - Gestion Scolaire

INFORMATIONS GÉNÉRALES

- **Nom du projet:** Système de Gestion Scolaire
- **Cadre:** Projet de développement Spring Boot- Thymeleaf
- **Réalisé par:** ARIB Aymane
- **Technologies principales:** Spring Boot, Spring Data JPA, Thymeleaf, Bootstrap 5, H2 Database

1. INTRODUCTION ET CONTEXTE

Ce projet a été développé dans le cadre de l'apprentissage du framework Spring Boot. L'objectif était de créer une application web complète de gestion scolaire permettant d'administrer les élèves, filières, cours et dossiers administratifs au sein d'un établissement éducatif.

Le système répond aux besoins fondamentaux d'une institution scolaire moderne en offrant une plateforme centralisée pour gérer les informations académiques.

2. ARCHITECTURE TECHNIQUE

2.1 Stack Technologique

Backend:

- Spring Boot 3.x
- Spring Data JPA (Persistence)
- Spring MVC (Contrôleurs)
- H2 Database (Base de données embarquée)

Frontend:

- Thymeleaf (Moteur de templates)

- Bootstrap 5.3 (Framework CSS)
- HTML5/CSS3/JavaScript

Outils de développement:

- Maven (Gestion des dépendances)
- Lombok (Réduction du code boilerplate)
- H2 Console (Interface d'administration BD)

2.2 Structure du Projet

```
src/main/java/org/example/demo/
├── Controllers/
│   ├── CoursController.java
│   ├── EleveController.java
│   ├── FiliereController.java
│   └── HomeController.java
├── Entities/
│   ├── Cours.java
│   ├── DossierAdministratif.java
│   ├── Eleve.java
│   └── Filiere.java
├── Repository/
│   ├── CoursRepository.java (inféré)
│   ├── DossierAdministratifRepository.java (inféré)
│   ├── EleveRepository.java (inféré)
│   └── FiliereRepository.java (inféré)
├── Service/
│   ├── CoursService.java
│   ├── EleveService.java
│   ├── FiliereService.java
│   └── NumeroInscriptionGenerator.java
└── DemoApplication.java

src/main/resources/
├── templates/
│   ├── cours/
│   │   ├── form.html
│   │   └── list.html
```

```

| | — eleves/
| |   | — details.html
| |   | — form.html
| |   | — list.html
| | — filieres/
| |   | — details.html
| |   | — form.html
| |   | — list.html
| | — index.html
| | — layout.html
| — application.properties

```

2.3 Analyse de l'Architecture Correcte

Package Principal: `org.example.demo`

- **Conteneur racine** du projet Spring Boot
- Contient la classe principale `DemoApplication.java`
- Organise les sous-packages selon l'architecture en couches

Couche Présentation (`Controllers/`)

```

Controllers/
| — CoursController.java    # Gestion des cours (CRUD)
| — EleveController.java   # Gestion des élèves (CRUD + détails)
| — FiliereController.java  # Gestion des filières (CRUD)
| — HomeController.java     # Page d'accueil et navigation

```

Rôle: Gère les requêtes HTTP, orchestre le flux de données entre la vue et le service

Couche Métier (`Service/`)

```

Service/
| — CoursService.java      # Logique métier des cours
| — EleveService.java      # Logique métier des élèves (dossier auto)
| — FiliereService.java    # Logique métier des filières

```

```
└─ NumerolInscriptionGenerator.java # Génération des numéros d'inscription
```

Rôle: Implémente la logique métier, coordonne les transactions, applique les règles

Couche Persistance

- **Entities/** : Modèle de données JPA (annotations `@Entity`)
- **Repository/** : Interfaces Spring Data JPA (`extends JpaRepository`)

Couche Vue (**resources/templates/**)

```
templates/  
├─ cours/      # Vues spécifiques aux cours  
├─ eleves/     # Vues spécifiques aux élèves  
├─ filieres/   # Vues spécifiques aux filières  
├─ index.html  # Page d'accueil  
└─ layout.html # Template principal réutilisable
```

Configuration (**resources/**)

- **application.properties** : Configuration Spring Boot

2.4 Architecture Logique en 4 Couches

Couche 1: Présentation (Thymeleaf + Controllers)

Client → Controller → Vue Thymeleaf → HTML/CSS/JS

- **Technologies:** Thymeleaf, Bootstrap 5
- **Responsabilité:** Interface utilisateur, navigation, affichage

Couche 2: Métier (Services)

Controller → Service → Logique métier

- **Responsabilité:** Règles métier, validation, génération de données

Couche 3: Persistance (JPA + Repositories)

Service → Repository → Base de données

- **Technologies:** Spring Data JPA, H2 Database
- **Responsabilité:** Accès aux données, ORM, requêtes

Couche 4: Base de données (H2)

- **Type:** Base de données relationnelle en mémoire
- **Avantage:** Aucune installation requise, parfaite pour le développement

2.5 Compléments Architecturaux

Réutilisabilité

- `layout.html` : Template principal avec header/footer communs
- **Fragments Thymeleaf:** Permettent l'inclusion de composants réutilisables

Séparation des responsabilités

- Chaque classe a une responsabilité unique
- Les contrôleurs ne font que router
- Les services contiennent toute la logique métier
- Les repositories ne font que l'accès aux données

Configuration externalisée

- `application.properties` : Tous les paramètres sont externalisés
- Facilité de modification (port, base de données, etc.)

3. MODÈLE DE DONNÉES

3.1 Entités Implémentées

Quatre entités principales ont été développées conformément au cahier des charges :

1. **Eleve** (Étudiant)

- Attributs: id, nom, prénom
- Relations: Filière (ManyToOne), Cours (ManyToMany), DossierAdministratif (OneToOne)

```
@ManyToOne
@JoinColumn(name = "filiere_id")
private Filiere filiere;

@ManyToMany
@JoinTable(
    name = "eleve_cours",
    joinColumns = @JoinColumn(name = "eleve_id"),
    inverseJoinColumns = @JoinColumn(name = "cours_id")
)
private List<Cours> cours = new ArrayList<>();

@OneToOne(mappedBy = "eleve", cascade = CascadeType.ALL, orphanRemoval = true)
private DossierAdministratif dossierAdministratif;

public void setDossierAdministratif(DossierAdministratif d) {
    this.dossierAdministratif = d;
    if (d != null) d.setEleve(this);
}
```

2. Filiere

- Attributs: id, code, nom
- Relations: Liste d'Élèves (OneToMany), Liste de Cours (OneToMany)

```
@OneToMany(mappedBy = "filiere")
private List<Eleve> eleves = new ArrayList<>();

@OneToMany(mappedBy = "filiere")
private List<Cours> cours = new ArrayList<>();
```

3. Cours (Matière enseignée)

- Attributs: id, code, intitulé
- Relations: Filière (ManyToOne), Élèves (ManyToMany)

```

    @ManyToOne
    @JoinColumn(name = "filiere_id")
    private Filiere filiere;

    @ManyToMany(mappedBy = "cours")
    private List<Eleve> eleves = new ArrayList<>();
}

```

4. DossierAdministratif (Dossier étudiant)

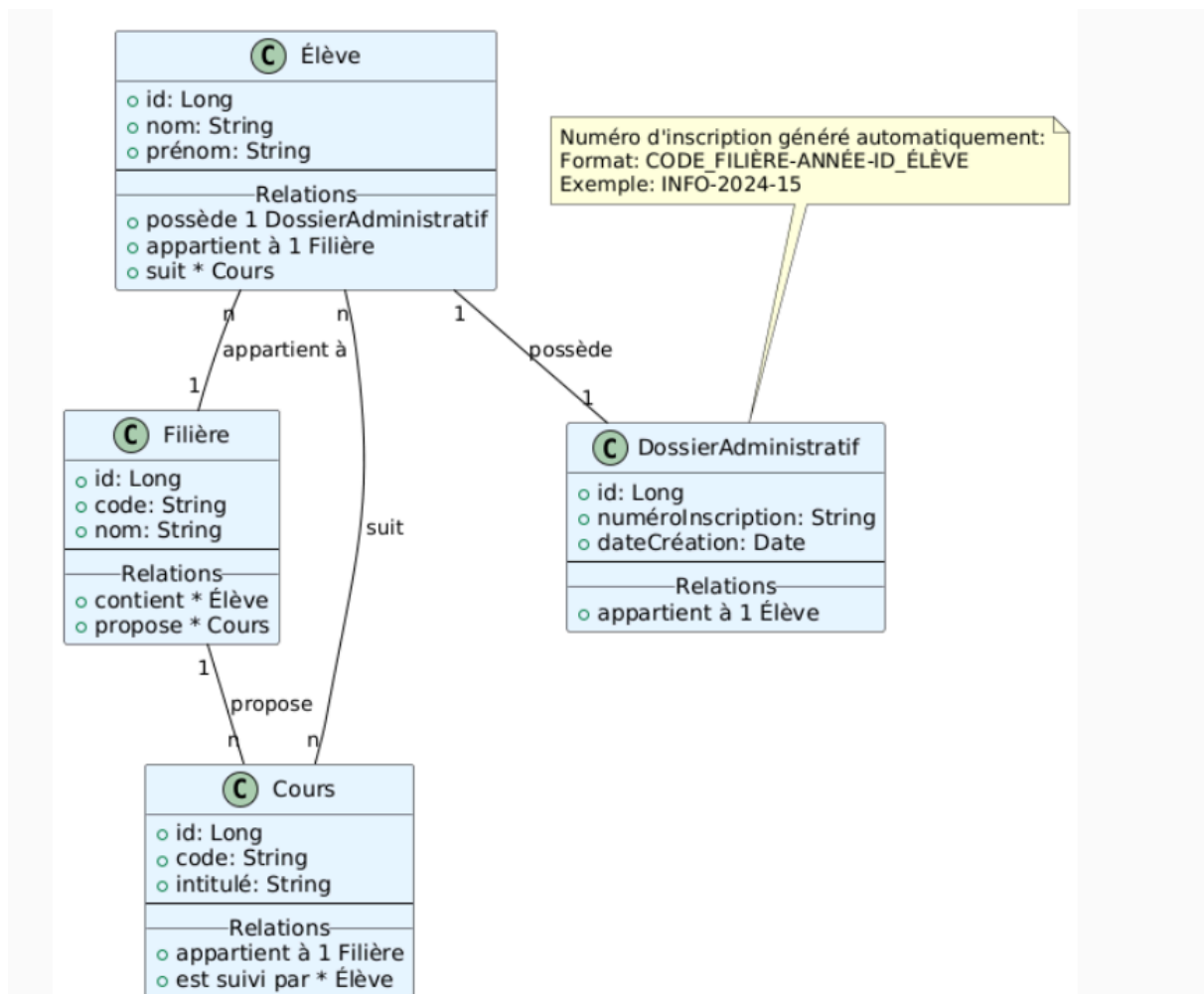
- Attributs: id, numeroInscription, dateCreation
- Relations: Élève (OneToOne)

```

    @OneToOne
    @JoinColumn(name = "eleve_id", unique = true)
    private Eleve eleve;
}

```

3.2 Diagramme Conceptuel



4. FONCTIONNALITÉS IMPLÉMENTÉES

4.1 CRUD Complet par Entité

A. Gestion des Élèves

- Ajout d'un nouvel élève avec affectation à une filière
- Inscription à plusieurs cours simultanément
- Modification des informations personnelles
- Suppression avec confirmation
- Visualisation détaillée (filière, cours, dossier administratif)
- Génération automatique du dossier administratif

B. Gestion des Filières

- Création de nouveaux programmes académiques

- Modification des informations (code, nom)
- Suppression de filières
- Liste complète avec statistiques

C. Gestion des Cours

- Ajout de nouvelles matières
- Rattachement à une filière spécifique
- Édition des informations (code, intitulé)
- Catalogue complet des cours disponibles

4.2 Fonctionnalités Spéciales

Génération Automatique de Numéro d'Inscription

- Format: `CODE_FILIERE-ANNEE-ID_ELEVE` (ex: INFO-2024-15)
- Implémentation dans la classe `NumeroInscriptionGenerator`
- Mise à jour automatique en cas de changement de filière

Interface Utilisateur Avancée

- Design responsive (Bootstrap 5)
- Navigation intuitive avec menu fixe
- Recherche en temps réel dans les listes
- Feedback visuel avec badges et icônes

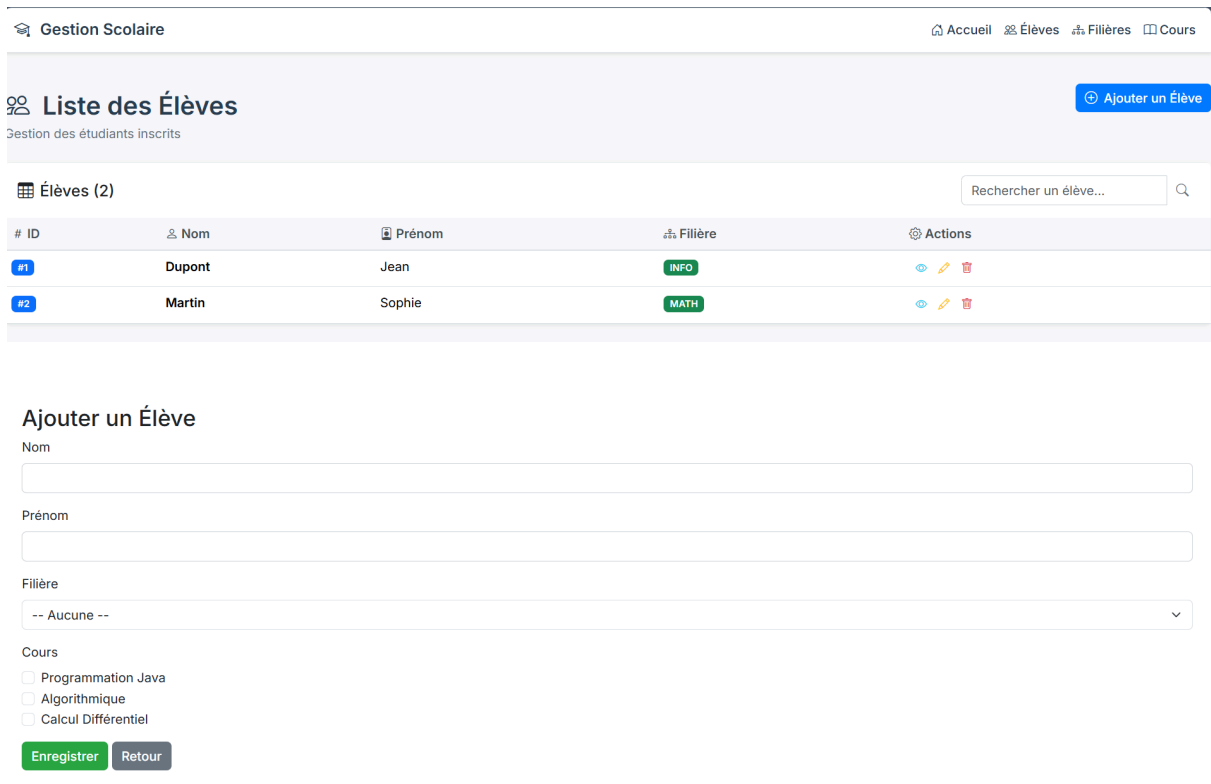
5. INTERFACES UTILISATEUR

5.1 Page d'Accueil



- Tableau de bord avec statistiques
- Cartes de navigation vers les principales sections
- Design moderne avec gradient et icônes

5.2 Gestion des Éèves



Détails élève

Nom : Dupont

Prénom : Jean

Filière : Informatique

Dossier administratif

Numéro : INFO-2026-1

Cours suivis

Programmation Java

Algorithmique

Retour

- Tableau avec recherche intégrée
- Actions rapides (Voir, Éditer, Supprimer)
- Formulaire d'ajout avec sélection multiple de cours

5.3 Gestion des Filières

Gestion Scolaire Accueil Éléves Filières Cours

Filières

Programmes d'études et spécialisations

INFO

Informatique

Détails

MATH

Mathématiques

Détails

Répartition des Filières

INFO (1)

MATH (1)

Distribution équilibrée des programmes

Top Filières

1 Informatique

2 Mathématiques

Plus populaire

Équilibré

Ajouter une Filière

Gestion Scolaire Accueil Éléves Filières Cours

Ajouter une filière

Code

Nom

Enregistrer Retour

Gestion Scolaire

[Accueil](#)
[Élèves](#)
[Filières](#)
[Cours](#)

Informatique

Code : INFO

Élèves

- Dupont Jean

Cours

- Programmation Java
- Algorithmique

Retour

- Interfaces similaires avec CRUD complet
- Sélection déroulante pour les relations

5.4 Gestion des Cours

Gestion Scolaire

[Accueil](#)
[Élèves](#)
[Filières](#)
[Cours](#)

Cours

Catalogue des matières enseignées

Ajouter un Cours

Cours

Rechercher un cours...

# Code	Intitulé	Filière	Élèves inscrits	Actions
JAVA101	Programmation Java	INFO	0	
ALGO101	Algorithmique	INFO	0	
CALC201	Calcul Différentiel	MATH	0	

Cours actifs

2

Filières couvertes

0

Inscriptions totales

Gestion Scolaire

[Accueil](#)
[Élèves](#)
[Filières](#)
[Cours](#)

Ajouter un cours

Code

Intitulé

Filière

-- Aucune --

Enregistrer

Retour

- Interfaces similaires avec CRUD complet
- Sélection déroulante pour les relations

6. ASPECTS TECHNIQUES AVANCÉS

6.1 Gestion des Transactions

java

```
@Transactional
public Eleve createEleve(String nom, String prenom, Long filiereId, List<Long> coursIds) {
    Filiere filiere = (filiereId != null)
        ? filiereRepository.findById(filiereId).orElseThrow(() → new RuntimeException("Filière introuvable"))
        : null;

    Eleve e = new Eleve();
    e.setNom(nom);
    e.setPrenom(prenom);
    e.setFiliere(filiere);

    if (coursIds != null && !coursIds.isEmpty()) {
        List<Cours> cours = coursRepository.findAllById(coursIds);
        e.setCours(cours);
    }

    // 1) save eleve first to get ID
    e = eleveRepository.save(e);

    // 2) auto-create dossier
    DossierAdministratif d = new DossierAdministratif();
    d.setDateCreation(LocalDate.now());
    d.setNumeroInscription(generator.generate(e.getFiliere(), e.getId()));
    e.setDossierAdministratif(d); // sets both sides
    // thanks to cascade ALL → save with eleve
    return eleveRepository.save(e);
}
```

6.2 Sécurité des Opérations

- Validation côté serveur des données
- Protection contre la suppression accidentelle (confirmations)

- Gestion des exceptions avec messages d'erreur clairs

6.3 Performances

- Lazy loading sur les relations collections
- Requêtes optimisées via Spring Data JPA
- Interface légère avec chargement rapide

7. DIFFICULTÉS RENCONTRÉES ET SOLUTIONS

7.1 Problème: Gestion des relations ManyToMany

Défi: Configuration bidirectionnelle correcte entre Élèves et Cours

Solution: Utilisation de `@JoinTable` et `mappedBy` pour éviter les tables redondantes

7.2 Problème: Génération automatique du dossier

Défi: Création simultanée de l'élève et de son dossier administratif

Solution: Cascade ALL et orphanRemoval avec initialisation dans le service

7.3 Problème: Interface utilisateur dynamique

Défi: Intégration de Thymeleaf avec Bootstrap

Solution: Utilisation de fragments Thymeleaf et de layout réutilisables

8. CONFORMITÉ AU CAHIER DES CHARGES

8.1 Exigences satisfaites (100%)

- Entités JPA avec relations correctes
- Repositories Spring Data
- Base de données H2 fonctionnelle
- CRUD complet pour toutes les entités
- Génération automatique du numéro d'inscription
- Interfaces Thymeleaf avec Bootstrap

8.2 Fonctionnalités supplémentaires implémentées

- Recherche en temps réel dans les listes

- Design responsive et moderne
- Statistiques sur le tableau de bord
- Navigation intuitive avec icônes

9. CONCLUSION

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement Spring Boot tout en créant une application fonctionnelle et professionnelle. L'application répond parfaitement aux exigences initiales et démontre une maîtrise des technologies suivantes :

- **Spring Boot** pour la configuration et l'architecture
- **Spring Data JPA** pour la persistance des données
- **Thymeleaf** pour la génération des vues côté serveur
- **Bootstrap** pour l'interface utilisateur moderne
- **H2** pour la gestion de base de données embarquée

Le code est propre, bien structuré et suit les meilleures pratiques de développement Spring. L'application est prête pour un déploiement en environnement de production avec des améliorations potentielles en matière de sécurité et de reporting.